MOTORFLOW

Informe Técnico: Configuración, Ejecución y Pruebas Unitarias

Proyecto: Ingeniería de Software 1 (Proyecto-IS1) | **Fecha:** Febrero 2026 | **Estado:** 100% Operativo y Verificado

---

✓ **Estado Global:** Tanto el backend de Spring Boot, como la base de datos MySQL (`tallerdb`), el frontend interactivo y la suite de **9 pruebas unitarias con JUnit 5** se encuentran completamente configurados y funcionando con un 100% de éxito.

1. Diagnóstico y Corrección en IntelliJ IDEA

Al intentar arrancar la aplicación por primera vez pulsando el botón verde de Play en `TallerApplication.java`, IntelliJ IDEA mostraba el mensaje de error: "*The file in the editor is not runnable (Mayús+F10)*".

Causa del problema:

- El proyecto abierto en IntelliJ era la carpeta raíz del repositorio (`Proyecto-IS1`), la cual contiene dos subcarpetas independientes: `backend` y `frontend`.
- IntelliJ no había detectado automáticamente el archivo `backend/pom.xml` como raíz de un módulo Maven, tratando los archivos Java como texto simple sin compilar y sin reconocer el método `main` como ejecutable.

Cambios aplicados y solución:

1. Se configuró el archivo `.idea/misc.xml` vinculando explícitamente `backend/pom.xml` a través del componente `MavenProjectsManager` y asignando el JDK 17 correspondiente.
2. Se creó la configuración del compilador en `.idea/compiler.xml` para el procesamiento de anotaciones de Spring Boot.
3. Se generó la configuración de ejecución en `.idea/runConfigurations/TallerApplication.xml`, permitiendo a IntelliJ habilitar de forma directa el botón de Play para la clase principal.

2. Verificación de la Base de Datos MySQL

Se revisó la conectividad con la base de datos configurada en `backend/src/main/resources/application.properties`:

- **Servicio activo:** El servicio de Windows `MySQL80` está en estado *Running*.
- **Base de datos:** La base de datos `tallerdb` ya existe en el servidor local (puerto `3306`).
- **Credenciales:** Usuario `root` y contraseña `admin123` autenticados correctamente.
- **Semilla de datos:** El componente `DatosPrueba.java` inserta de forma automática los usuarios, vehículos, materiales y reparaciones de prueba en el primer arranque.

3. Ejecución del Frontend y Conexión con el Backend

El frontend de MotorFlow está construido con HTML, CSS y JavaScript nativo, lo que permite abrirlo de forma inmediata sin necesidad de instalar dependencias de Node.js o librerías pesadas.

Cómo abrir el frontend:

- **Opción recomendada:** Abrir el archivo `frontend/index.html` en IntelliJ y hacer clic en el icono flotante del navegador (Chrome / Edge) en la esquina superior derecha del editor.
- **Alternativa:** Hacer doble clic directamente sobre `frontend/index.html` en el explorador de archivos de Windows.

Cuentas de usuario disponibles para pruebas:

Rol	Correo electrónico	Contraseña	Funcionalidad principal
Administrador	admin@taller.com	admin123	Control total: gestión de empleados, inventario de piezas, confirmación final de reparaciones y caja del taller.
Recepcionista	laura@taller.com	laura123	Recepción de coches, alta de nuevos clientes, apertura de averías y asignación inicial.
Mecánico	pedro@taller.com	pedro123	Visualización de órdenes asignadas, registro de piezas usadas del almacén y horas de mano de obra.
Cliente	juan@email.com	juan123	Seguimiento en tiempo real de su vehículo mediante línea temporal interactiva y consulta de facturas.

4. Las 9 Pruebas Unitarias del Sistema (JUnit 5)

Siguiendo las buenas prácticas de la programación orientada a objetos (probar las clases y métodos como unidades independientes), se han implementado **9 pruebas unitarias** en el archivo: `backend/src/test/java/com/taller/PruebasUnitariasTest.java`

Nº Test	Clase y Método	Objetivo y Comprobación	Aserciones utilizadas
Prueba 1	`Factura recalcularImportes()`	Comprueba que al registrar materiales (100 €) y 2 horas de mano de obra (70 €), se calculan exactamente el margen del taller (15% = 25,50 €), el IVA (21% = 41,06 €) y el total final (236,56 €).	`assertEquals(esperado, obtenido, delta)`
Prueba 2	`Factura recalcularImportes()`	Comprueba que cuando no se consumen piezas del almacén (coste 0,00 €), el método liquida únicamente las horas de mano de obra sin fallos de cálculo.	`assertEquals(0.0, costeMateriales)` `assertEquals(48.70, totalFactura)`
Prueba 3	`PresupuestoReparacion recalcularTotales()`	Verifica que el presupuesto suma correctamente las líneas agregadas, diferenciando el subtotal de piezas respecto al subtotal de mano de obra.	`assertEquals(50.00, subtotalPiezas)` `assertEquals(60.00, subtotalManoObra)`
Prueba 4	`PresupuestoReparacion estaCaducado()`	Verifica la lógica temporal: un presupuesto con fecha futura devuelve *falso* (vigente) y uno con fecha anterior a hoy devuelve *verdadero* (caducado).	`assertFalse(estaCaducado())` `assertTrue(estaCaducado())`
Prueba 5	`Reparacion generarLocalizador()`	Verifica que el generador automático de códigos de seguimiento produce una cadena no nula con una longitud exacta de 8 caracteres alfanuméricos.	`assertNotNull(localizador)` `assertEquals(8, localizador.length())`
Prueba 6	`Material Constructor y Getters`	Valida la correcta inicialización y consistencia de los datos de un material del inventario (nombre, descripción, stock físico y precio unitario).	`assertEquals("Filtro...", nombre)` `assertEquals(15, stockDisponible)`
Prueba 7	`UsuarioService login()`	**Inicio de sesión exitoso:** Comprueba que un empleado con email y contraseña correctos inicia	`assertNotNull(response)` `assertEquals("MECANICO", rol)`

Nº Test	Clase y Método	Objetivo y Comprobación	Aserciones utilizadas
		sesión, recibe su rol (ej: MECANICO) y el mensaje de bienvenida.	
Prueba 8	`UsuarioService login()`	**Rechazo por contraseña errónea:** Comprueba que si se introduce una contraseña equivocada, el método rechaza el acceso lanzando `IllegalArgumentException` ("Contraseña incorrecta.").	`assertThrows(IllegalArgumentException.class, ...)`
Prueba 9	`Cliente tieneCuenta()`	**Validación de cuenta cliente:** Verifica que el método distingue si un cliente tiene credenciales de acceso para la web (email y password) o si solo dejó el coche en el taller.	`assertFalse(clienteSinCuenta)` `assertTrue(clienteConCuenta)`

5. Guía de Ejecución y Validación de las Pruebas

A. Ejecución desde IntelliJ IDEA:

1. En el árbol de carpetas, navega hasta `backend > src > test > java > com.taller`.
2. Haz clic derecho sobre el archivo **`PruebasUnitariasTest.java`**.
3. Selecciona **Run 'PruebasUnitariasTest'** (o pulsa Mayús+F10).
4. Se abrirá la pestaña inferior de pruebas de IntelliJ mostrando los 9 tests con el indicador verde de éxito.

B. Ejecución por consola con Maven:

Desde una terminal (PowerShell o CMD) situada en la carpeta `backend`, ejecuta:

```
.\apache-maven-3.9.6\bin\mvn test
```

Resultado obtenido en consola:

```
------------------------------------------------------- T E S T S -------------------------------------
-------------- Running com.taller.PruebasUnitariasTest Tests run: 9, Failures: 0, Errors: 0, Skipped: 0,
Time elapsed: 0.919 s -- in com.taller.PruebasUnitariasTest Results: Tests run: 9, Failures: 0, Errors: 0,
Skipped: 0 [INFO] ------------------------------------------------------------------------ [INFO] BUILD
SUCCESS [INFO] Total time: 4.550 s [INFO] ------------------------------------------------------------------------
-------
```

Nota sobre el código añadido: Se ha actualizado también el archivo de texto plano `CODIGO_PRUEBAS_UNITARIAS.txt` en la raíz del proyecto para que puedas copiar, revisar y entregar las 9 pruebas unitarias completas.